

Plataforma de Vigilância Epidemiológica - Resumo Executivo

Pipeline de dados end-to-end para ingestão, processamento e análise de notificações epidemiológicas do DataSUS, usando arquitetura medallion (Bronze -> Silver -> Gold) sobre Delta Lake em Kubernetes.

Autor: Matheus Fideles Martins de Souza 20/07/2026

GitHub: <https://github.com/Matheus-Fideles/case-data-master-data-engineer>

17

DAGs Airflow

3

Camadas Medallion

7

Tabelas Gold

106

Testes Unitários

36

Testes de Fumaça

1. Contexto e Objetivo

O Brasil notifica mais de 1,2 milhão de óbitos anuais via SIM (Sistema de Informação sobre Mortalidade) e dezenas de milhares de internações via SIH. Esses dados, disponibilizados pelo DataSUS em formato FTP/CSV, carecem de uma camada analítica acessível para gestores de saúde tomarem decisões em tempo hábil.

Este case demonstra a construção de uma plataforma de dados end-to-end capaz de:

- Ingerir dados epidemiológicos brutos do DataSUS (SIM, SIH) via API REST e SFTP offline.
- Aplicar mascaramento de PII (CPF/nome) com SHA-256 + salt, em conformidade com a LGPD.
- Transformar e qualificar os dados em arquitetura medallion (Bronze -> Silver -> Gold).
- Exportar tabelas Gold via Trino/Delta Lake para consultas analíticas em Metabase.
- Monitorar toda a plataforma com Prometheus, Grafana e rastreabilidade de linhagem via Marquez/OpenLineage.

2. Arquitetura da Solução

A plataforma adota o padrão Medallion em três camadas sobre Delta Lake armazenado no MinIO (S3-compatible). O processamento é feito por Spark 3.x rodando como SparkApplication CRDs em k3s (Rancher Desktop), orquestrado pelo Airflow via SparkKubernetesOperator.

Bronze

Dados brutos do DataSUS. Schema On Read. Idempotente por partição data_ref.

`s3a://bronze/hospitalar/{fonte}/{data_ref}/`

Silver

PII mascarada. Cast de tipos. Datas corrigidas (CORRECTED parser). Delta particionado.

`s3a://silver/hospitalar/{fonte}/`

Gold

Star schema dimensional. KPIs calculados. Partição por ano/mes. Prontos para BI.

`s3a://gold/hospitalar/{tabela}/`

3. Stack Tecnológica

Componente	Tecnologia	Versão	Papel
Orquestração	Apache Airflow	2.9.3	17 DAGs; SparkKubernetesOperator
Processamento	Apache Spark	3.5	SparkApplication CRDs em k3s
Data Lake	Delta Lake + MinIO	3.x / 2024-07	Storage S3-compatible + ACID
Catálogo	Hive Metastore	3.1.3	Metastore para Trino
Query Engine	Trino	448	SQL federado sobre Delta
BI	Metabase	0.51.4	Dashboards para gestores
Mensageria	Apache Kafka	7.6.1 (KRaft)	Tópico notificacoes.raw
Linhagem	Marquez / OpenLineage	0.47.0	Rastreabilidade de datasets
Monitoramento	Prometheus + Grafana	2.52 / 10.4	Métricas infra e pipeline
Kubernetes	k3s (Rancher Desktop)	1.28	Cluster local para Spark
IaC	Docker Compose	v2	4 perfis: core/streaming/serving/obs

4. Decisões de Arquitetura (ADRs)

ADR	Decisão	Justificativa
ADR-0001	Spark como motor de transformação	Performance, Delta Lake nativo, PySpark testável
ADR-0002	Delta Lake como formato de armazenamento	ACID, time-travel, schema evolution
ADR-0003	OFFLINE_MODE para desenvolvimento	Reprodutibilidade sem depender do DataSUS
ADR-0004	Mascaramento SHA-256 + salt	LGPD Art. 12 - dado anonimizado não é dado pessoal
ADR-0005	Medallion em 3 camadas	Separação de responsabilidades, reprocessamento seguro
ADR-0006	Airflow como orquestrador	Retry nativo, SLA tracking, integração OpenLineage
ADR-0007	Spark on Kubernetes (k3s)	Isolamento de recursos, submissão declarativa via CRD
ADR-0008	Trino sobre Delta Lake	SQL padrão sem mover dados, federação multi-catálogo
ADR-0009	OpenLineage / Marquez	Rastreabilidade de linhagem, auditoria de datasets

5. Pipeline de Dados - 17 DAGs

Bronze (Ingestão)

- dag_bronze_sim_obitos - SIM: óbitos DataSUS
- dag_bronze_sih_aih - SIH: internações (AIH)
- dag_bronze_sivep_gripe - SIVEP: vigilância gripal
- dag_bronze_sinan_dengue - SINAN: notificações dengue
- dag_bronze_ibge_populacao - IBGE: estimativas populacionais
- dag_bronze_notificacoes_stream - Kafka consumer -> Bronze

Silver (Qualidade & PII)

- dag_silver_sim_obitos - Parse datas, máscaras PII, tipos
- dag_silver_sih_aih - Normalização procedimentos/diagnósticos
- dag_silver_sivep_gripe - Enriquecimento semana epidemiológica
- dag_silver_sinan_dengue - Dedup, tipagem, coordenadas

Gold (Analítico)

- dag_gold_dim_tempo - Dimensão calendário epidemiológico
- dag_gold_dim_cid - Dimensão CID-10
- dag_gold_dim_municipio - Dimensão IBGE municípios
- dag_gold_dim_estabelecimento - Dimensão CNES
- dag_gold_fato_obito - Fato óbitos (SIM × dims)
- dag_gold_fato_internacao - Fato internações (SIH × dims)
- dag_gold_kpi_mortalidade - KPI taxa mortalidade por 100k hab

Qualidade

- dag_quality_checks - Great Expectations: contratos de dados

6. Qualidade e Testes

A suite de testes cobre todas as camadas do pipeline com isolamento por tipo:

Tipo	Qtd	Escopo
Unitários (pytest)	106	Transformações Spark, mascaramento PII, parsers
Fumaça (smoke)	36	Conectividade infra: MinIO, Postgres, Airflow, Marquez
Qualidade (GX)	-	Contratos de schema, null rates, distribuições
Segurança (bandit)	0 issues	SAST estático em apps/ e airflow/

CI/CD: GitHub Actions executa ruff (format + lint), pytest, bandit e build de imagens a cada push. Todos os 142 testes passam no pipeline (commits 3784c23 e f2ef66b).

7. Governança e LGPD

O tratamento de dados pessoais segue a Lei Geral de Proteção de Dados (Lei 13.709/2018). A estratégia de anonimização aplicada é SHA-256 com salt variável, garantindo que o dado resultante não permita reidentificação direta (Art. 12 LGPD).

- PII_SALT armazenado em Kubernetes Secret - nunca em código ou variáveis estáticas.
- Mascaramento aplicado na camada Silver, antes de qualquer escrita persistente.
- Campos afetados: CPF, nome_paciente, nome_mae - substituídos por hash hex de 64 chars.
- Acesso ao MinIO controlado por credenciais dedicadas (MINIO_ROOT_USER/PASSWORD).
- Audit trail via OpenLineage: toda transformação gera evento de linhagem rastreável.
- Dados brutos (Bronze) acessíveis apenas ao pipeline - sem exposição via SQL/BI.

8. Observabilidade

A stack de observabilidade cobre métricas de infraestrutura, pipeline e data quality:

Serviço	Função
Prometheus :9090	Coleta métricas de cAdvisor (containers), MinIO (buckets/objetos) e self-monitoring
Grafana :3000	Dashboard Pipeline Overview: CPU, memória, rede, capacidade MinIO, status UP/DOWN
cAdvisor :8088	Exporta métricas Docker em tempo real para Prometheus
Marquez Web :5012	UI de linhagem OpenLineage - grafo de datasets e jobs
Airflow :8080	UI de orquestração - status de DAGs, logs de tarefas, SLA tracking
Kafka UI :8090	Monitoramento de tópicos, consumer groups e lag

9. Como Executar

```
git clone https://github.com/Matheus-Fideles/case-data-master-data-engineer
cp .env.example .env # configurar credenciais
make up-core         # Postgres + MinIO + Airflow
make up-all          # todos os profiles (streaming + serving + observability)
# Acesse: Airflow :8080 . MinIO :9001 . Grafana :3000 . Marquez :5012
# Trigger: Airflow UI -> dag_bronze_sim_obitos -> trigger w/ config
make test            # 106 unit + 36 smoke tests
```

10. Repositório e Contato

GitHub: <https://github.com/Matheus-Fideles/case-data-master-data-engineer>
Autor: Matheus Fideles Martins de Souza
E-mail: matheuss.fideles@hotmail.com
Data: 20/07/2026